

MEMORANDUM

To: D. Walden, W. Crowther, R. Alter, J. Cole, E. Belove

From: B. Cosell

Subject: PDP-1 User Interface to the Network

Date: 21 April 1972

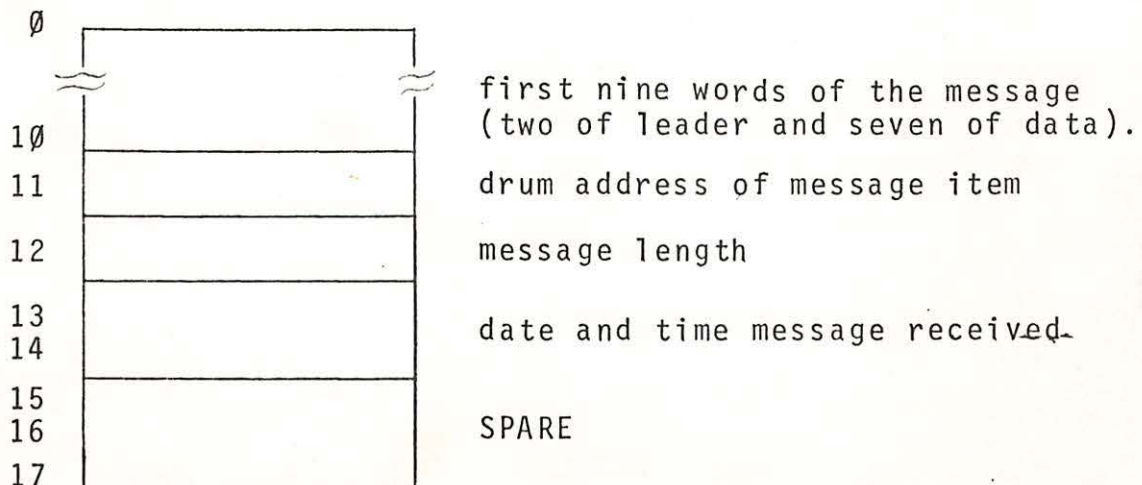
This memo reflects my current thoughts about the design of the User Interface to the network. The memo is not in any way intended to be "user oriented", but rather is intended to be a system level discussion of the mechanism. A package for user access to the network is a different problem and I will worry about it later.

I see three major constraints governing the design of the User Interface:

1. Exec modifications must be kept to a minimum.
2. The interface must be very flexible.
3. There is no facility within the Exec for inter-program communication.

To preserve flexibility, the Exec will do no preprocessing of data either to or from the network. This, unfortunately, means that each user program will have to handle its own IMP/Host protocol, as well as its own Host/Host protocol. Communication between the Exec and network user programs will be effected by means of queues on the Fastrand.

All messages that come in from the network will get forwarded onto the "network input queue" which will consist of a single item addressed by an invariant number. The item will consist of a series of 16 word blocks, one per message. For each message, its block in the queue will contain:



Messages of nine words or less will have no separate item associated with them. Both the queue and all messages will reside on third two.

To keep the drum from slowly filling up with messages that no user program wants, there will be a special user program whose purpose will be to garbage collect this queue. A message block will be removed from the queue, and the message it addresses removed from the drum, when the message is more than one minute old. Also, to make it a little easier for user programs to do their own garbage collection, it will remove any message whose drum address entry in the queue has its sign bit set. Thus, a user program would look at a message and when done with it just set the sign bit in the drum address word of its message's block, and the garbage collector will remove the block from the queue and the message from the drum. If the user program would like to retain the message, it would rewrite the drum address entry in its message's block with a sign bit and all zeroes, that is, "delete this message block but no item is associated with this message."

For output to the network, the procedure is similar. There will be a queue on third two addressed by an invariant number. Users will place the drum addresses of messages for the network onto this queue. The addresses will be removed from the queue when the message is sent to the network, and, once again to simplify garbage collection for the user program, if the message's address was placed on the queue without its sign bit set, the message itself will be expunged from the drum when it is sent.

In the event that the IMP is down or that the IMP is being used for some purpose which requires the exclusion of other user programs, the invariant numbers pointing to the input and output queues will both be set to zero. This will be the signal to user programs that they may not have access to the network.

Other Problems and Considerations

This scheme is not quite general enough for some of the uses we will be making of the network. In particular, it will be difficult to write user programs which conform to the current Host protocol. A single message from the net over the control link may contain different pieces to be distributed among different programs, and user programs transmitting over the control link cannot know if some other user program is using it at the time. Therefore, the Exec will break up messages over link one from the

Memorandum
Page 3
21 April 1972

network into individual control messages so that they can be found by the programs awaiting them, and it will also maintain a "blocked control link" table for governing transmissions to the network.

Another problem is that for almost every use of the network, we will have to invent global mechanisms for assigning unique numbers, be they for links, ports, sockets, or whatever.

Looking over the scheme, I feel that I may have gone overboard in sacrificing user convenience and speed in favor of simplicity in the Exec. Within bounds, the balance can be shifted somewhat. I am relying on you to help with ideas on how to do this. Please forward to me any suggestions or improvements you can come up with.

Memorandum
Page 4
21 April 1972

APPENDIX: Sample User subroutines

These subroutines are not several things. They are not debugged; they have never even been assembled. They are not my thoughts as to what the user package for accessing the network should be. They do not include IMP/Host Protocol.

They are — 1) to find out how unwieldy the mechanism is (~ 1000 words); 2) to provide an example of how to deal with shared structures on the PDP-1; and 3) to learn, from the readily apparent inadequacy of these subroutines, what the correct set of subroutines for the user package is.

BC/jm
Attach.

/SAMPLE SUBROUTINES FOR ACCESSING THE NETWORK VIA THE QUEUES

/SUBR "SEARCH" - SCANS INPUT QUEUE LOOKING FOR MESSAGES FOR
/ THIS USER
/USER SHOULD SUPPLY SUBROUTINE "CHECK" WHICH SKIPS IF "INBLK"
/ IDENTIFIES A MESSAGE FOR HIM
/GIVES R1 IF THE IMP IS DOWN
/GIVES R2 IF NOTHING ON THE QUEUE IS OF INTEREST
/GIVES R3 WITH DRA OF MSG IN I/O, MSG ID IN "INBLK"

```
SEARCH,    DAP SRCHX
SRCH3,     LIO (NETING)  /GET DRA OF THE QUEUE
           IWR
           DIO INQDRA
           SNI
           JMP SRCHX I    /IMP IS DOWN
           CLA           /SET TO BEGIN AT START OF QUEUE
SRCH2,     DAC SKIP
           LAW SGCMD5     /GET NEXT MSG ID
           LIO INQDRA
           SGI+5
           JMP RDERR
           LAC MSGDRA
           SPA           /THIS BLOCK REALLY THERE?
           JMP SRCH6      /IT'S WAITING TO BE DELETED
           JSP CHECK
           SKP I
           JMP SRCH1      /FOUND A GOOD ONE
SRCH6,     LAW 10.        /LOOP TO NEXT MSG ID
           ADD SKIP
           JMP SRCH2

SRCH1,     LIO MSGDRA
           IDX SRCHX
SRCH4,     IDX SRCHX
SRCHX,     JMP .

INQDRA,    0
```

KDERR,

LAC ERCODE
SAD (2000)
JMP SRCH3
SAD (40000)
JMP SRCH5
HLT
JMP SRCH3

/LOOKS LIKE QUEUE MOVED (MAYBE BY GARBAGE COLLECTOR)
/LOOKS LIKE WE'VE HIT END OF THE QUEUE
/BAH?
/GO TRY AGAIN IF CONTINUE HIT

SRCH5,

LAC QLEN
SUB SKIP
SPA
JMP SRCH4
HLT
JMP SRCH3

/ARE WE REALLY AT END?
/REALLY AT END - SO NOTHING FOR US
/BAH??

QLEN, 0
RWITNM, 0
INBLK, .+9./
MSGDRA, 0
INBLK+16./

/LENGTH OF QUEUE
/QUEUE'S REWRITE NUMBER
/MSG ID GOES HERE

SGCMDS,

QLEN
2
1
.
INBLK
16.
-0

/READ LENGTH AND REWRITE NUMBER
/NOW SKIP STUFF WE'VE LOOKED AT ALREADY
/NOW READ MESSAGE ID

SKIP,

REMOVE,	DAP RMVX CLA JMP RMV1	/REMOVE ADDRESSED BY INBLK FROM QUEUE
EXPUNJ,	DAP RMVX LAC MSGDRA	/HAVE GC EXPUNGE MESSAGE FROM DRUM
RMV1,	IOR (400000) DAC RMVVAL LAC OWNWD DAC SVOWN LAC (FLEXO IMP) DAC OWNWD LAC SKIP DAC RRSKIP ADD (MSGDRA-INBLK) DAC RWSKIP	/SAVE UP USER'S OWNWD /SET UP OF QUEUE'S /SET UP TO GET TO PROPER PLACE IN THE QUEUE
RMV2,	LAW RWCMDs LIO INQDRA SGI+5 JMP RMVERR	/TRY TO REWRITE THE BEAST
RMVRET,	LAC SVOWN DAC OWNWD	/PUT BACK OWN WORD
RMVX,	JMP .	
RMVERR,	LIO (NETING) IVNR SVI JMP RMVRET DIO INQDRA LAW RMVRRD SGI+5 JMP RMVRET JMP RMV2	/IMP HAS GONE DOWN /TRY TO CHECK IF OUR MSG IS STILL THERE /GUESS NOT, SO WHAT MORE CAN WE DO?? /OK, TRY AGAIN WITH NEW REWRITE NUMBER
SVOWN,	0	/USER'S OWNWD SAVED HERE
RMVVAL,	0	/ADDRESS TO PUT INTO QUEUE
RWCMDs,	IOR QLEN 2 1	/RE- WRITE LENGTH AND REWRITE # /LEAP UP TO WHERE OUR DRA SHOULD BE
RWSKIP,	. IOR RMVVAL 1 -0	/AND WRITE CORRECT FLAVOR OF REMOVED DRA
RMVRRD,	QLEN 2 1	/RE-READ LENGTH AND REWRITE # /LEAP UP TO OUR MSG BLOCK
RRSKIP,	. AND INBLK 16. -0	/NO ERROR IF OUR MESSAGE IS STILL THERE

*PUT A MESSAGE'S DRA ON OUTPUT QUEUE TO THE NET
 /JDA'D TO WITH DRA OF MESSAGE IN AC
 /R1 => IMP IS DOWN
 /R2 => MESSAGE SENT

FORNET,	0	
	DAP RMVRET	
	LAC OWNWD	/SAVE USER'S OWNWD
	DAC SVOWN	
	LAC (FLEXO IMP)	
	DAC OWNWD	
FN1,	LIO (NETOTQ)	/GET DRA OF OUTPUT QUEUE
	IVNR	
	SNI	
	JMP RMVRET	/IMP IS DOWN
	DIO OTQDRA	
	LAW FORCMD	
	SGI+5	/GET LENGTH AND REWRITE NUMBER
	JMP FN1	/ONLY IMP DOWN CAN NAIL THIS ONE
	LAW I 2	/SET TO SKIP DATA CURRENTLY IN QUEUE
	ADD FORVAR	
	DAC FORSKP	
	IDX FORVAR	/AND BUMP LENGTH
	LAW FORRW	
	SGI+5	
	JMP FN1	/IMP'S DOWN OR SOMEBODY CHANGED IT UNDER US
	IDX RMVX	
	JMP RMVRET	/DONE
FORCMD,	FORVAR	/READ LENGTH AND REWRITE NUMBER
	2	
	-0	
FORRW,	IOR FORVAR	/RE-WRITE LENGTH AND REWRITE NUMBER
	2	
	I	/SKIP TO END OF ITEM
FORSKP,	.	
	IOR FORNET	/CRAM OUR MSG ONTO THE END
	1	
	-0	
FORVAR,	0	0
OTQDRA,	0	

CONSTANTS
 START